

EcoGraph — Smart Waste Collection Routing System

Project Proposal — C++ Backend Redesign

Problem Description

Municipal waste collection routes are often static and inefficient, with trucks following fixed paths regardless of the actual shortest route between homes, bins, and the depot. This wastes fuel, time, and labor. EcoGraph models a city's waste-collection network as a weighted graph — comprising a depot, homes, road junctions, and disposal bins — and computes optimal collection routes using shortest-path algorithms.

This project builds the system with C++ as the primary backend: a persistent HTTP server that owns the graph, performs routing and waste classification in-process, and serves JSON directly to a JavaScript frontend.

Features

- Graph-based city model — depot, homes, junctions, and bins represented as weighted graph nodes and edges
- Shortest-path routing using Dijkstra's algorithm for depot-to-home and home-to-bin segments
- Two collection strategies, selectable via a toggle: Priority mode (dispose after each pickup: home → bin → next home → bin...) and FIFO mode (collect from all queued homes first, then dispose in sequence)
- Rule-based waste classification — keyword matching (e.g. plastic, paper, organic) to route waste to the correct bin type
- REST API exposing endpoints for graph data, route computation, and waste classification
- Live route animation on the frontend, showing the truck traversing the computed path

Concepts Used

Concept	Application in the Project
Graph (adjacency list)	City map: nodes represent depot/homes/junctions/bins; edges represent weighted travel costs
Dijkstra's algorithm + min-heap priority queue	Computes the shortest path from the depot or a home to the next stop
Queue (FIFO)	Maintains the order in which homes are visited in FIFO mode
Path reconstruction (vector/stack)	Backtracks through Dijkstra's predecessor map to build the final route
Hash map (unordered_map)	Maps node IDs to adjacency lists and graph coordinates for O(1) lookup
String matching / rule engine	Classifies waste type from keyword descriptions
HTTP server and REST routing	Serves the frontend and exposes the routing/classification API
JSON serialization	Structures request and response bodies between backend and frontend

Recommended Technology Stack

- C++ HTTP framework: cpp-httplib (single-header, minimal setup) or Crow (Express.js-style routing)
- JSON library: nlohmann/json (header-only)
- Frontend: Cytoscape.js for graph rendering and route animation

Implementation Steps

1. Define the Graph Data Model

Create Node and Edge structures (id, type, coordinates, weight) and a Graph class backed by an `unordered_map` adjacency list. Load the city layout — depot, homes, junctions, bins — from a JSON configuration file at startup.

2. Implement Dijkstra's Algorithm

Write a `shortestPath(Graph&, source, destination)` function using a min-heap priority queue. Return both the path (sequence of node IDs) and the total travel cost. Validate against known shortest paths in the sample graph.

3. Implement Waste Classification

Write a `classifyWaste(description)` function using keyword matching to map a waste description to a bin type (e.g. recyclable, compost, general).

4. Implement the Two Routing Modes

Priority mode: for each home in order, route depot/previous-bin → home → nearest matching bin, concatenating segments. FIFO mode: route depot → home1 → home2 → ... → homeN, then bin1 → bin2 → ..., visiting all pickups before any disposal.

5. Build the HTTP Server

Expose three endpoints: GET `/graph` (returns the full node/edge list as JSON), POST `/route` (accepts a home list and mode, returns the computed path and cost), and POST `/classify` (accepts a waste description, returns the bin type).

6. Serve the Frontend

Serve static frontend files (HTML/CSS/JS) directly from the C++ server, so the API and frontend share the same origin.

7. Connect the Frontend Toggle and Animation

On route computation, the frontend sends the selected homes and mode to POST `/route` and animates the returned path using Cytoscape.js — pulsing nodes and highlighting edges in sequence.

8. Add Concurrency and Persistence (Stretch Goal)

Introduce a thread pool for concurrent request handling, and optionally persist graph and queue state to a JSON file or SQLite database across restarts.